

Programación en Python utilizando NumPy, Pandas y Matplotlib

Laboratorio IV.2

Análisis de datos de telemetría de un nodo IoT

Planteamiento del problema:

Un nodo de sensores IoT, alimentado por un pequeño panel solar, registra cada 15 minutos su temperatura ambiente, la humedad relativa, el voltaje de su batería y la potencia de señal (RSSI) de su enlace inalámbrico. Estas mediciones se almacenan en el archivo `telemetria_nodo_iot.csv`, que contiene 1000 registros.

Se deberá desarrollar un programa en Python que integre *numpy*, *pandas* y *matplotlib* para cargar, analizar, visualizar y resumir estos datos, generando como resultado un archivo Excel con un resumen diario de la telemetría.

Descripción del archivo entregado `telemetria_nodo_iot.csv`:

Columna	Descripción	Unidad
<i>timestamp</i>	Fecha y hora del registro	—
<i>temperatura C</i>	Temperatura ambiente medida por el nodo	°C
<i>humedad pct</i>	Humedad relativa del ambiente	%
<i>voltaje bateria V</i>	Voltaje de la batería del nodo alimentado por un panel solar	V
<i>rss_i dbm</i>	Potencia de la señal recibida en el enlace inalámbrico del nodo	dBm

Consiga:

1. Cargar el archivo CSV con *pandas*, interpretando correctamente la columna *timestamp* como fecha y hora, y utilizarla como índice del *DataFrame*.
2. Calcular, con *pandas* y *numpy*, las estadísticas descriptivas (media, mínimo, máximo y desvío estándar) de cada variable numérica.
3. Definir un criterio de alerta a partir de *arrays* de *numpy* e indexado booleano: por ejemplo, `voltaje_bateria_V < 3.5 V` (batería baja) o `rss_i_dbm < -85 dBm` (señal débil). Contar cuántos registros cumplen cada condición y cuántos presentan al menos una alerta.
4. Generar con *matplotlib* al menos un gráfico que muestre la evolución temporal de dos variables (por ejemplo, temperatura y voltaje de batería), señalando además los momentos en los que se detectó alguna alerta.
5. Agrupar los datos por día con *pandas* y calcular un resumen que incluya, como mínimo: temperatura promedio/máxima/mínima, voltaje de batería promedio/mínimo y cantidad de alertas por día.
6. Exportar ese resumen diario a un archivo `.xlsx` utilizando *pandas* (*DataFrame.to_excel*), en una hoja llamada "Resumen diario".

Subir al AV un archivo comprimido que contenga:

- El script de Python (.py).
Dicho archivo deberá contener, en forma de comentario, la URL del repositorio público de Github utilizado durante el desarrollo del laboratorio. En él deberán verse reflejadas las intervenciones (commits) de todos los integrantes del grupo.
- El archivo .xlsx generado, con el resumen diario de la telemetría.

Nota: La librería *openpyxl* es requerida por *pandas* para poder escribir archivos .xlsx

Instalación: *pip install openpyxl*

Rúbrica de evaluación

Criterio	Puntaje
1. Lectura y preparación de datos con <i>pandas</i>	2
2. Cálculos y detección de alertas con <i>numpy</i>	2
3. Visualización con <i>matplotlib</i> .	2
4. Resumen diario y exportación a xlsx	2
5. Calidad y organización del código	2
Total=	10